

Day 2

Anatomy of an LLM Call

Everything you do when typing into ChatGPT's "Ask anything" box is an **LLM call** — just done via code/API here.

The 4 required parts of a call

#	Part	What it is	ChatGPT analogy
1	API Key	A password that registers you	Signing up / subscription
2	Client	You are the client; Groq is the server	Opening the ChatGPT site
3	Model	The exact model name (one wrong letter = fail)	Choosing a model from the dropdown
4	Messages	What you ask (role + content)	Typing in the "Ask anything" box

We use **Groq** because it's free. (Ollama runs locally but won't run well on every laptop.)

🍏 **Mac note:** Ollama actually runs *very well* on Apple Silicon (M1–M4). But we stick with Groq (cloud) so everyone has an identical setup.

Messages = role + content (the key concept)

Every message has two things: a `role` and its `content`.

Three roles exist:

- `user` → you (the default).
- `assistant` → the LLM's reply.
- `system` → tells the LLM how to behave (covered Day 3).

Why roles matter — Context. Consider:

```
User:      "Who is Virat Kohli?"
Assistant: "Virat Kohli is an Indian cricketer."
User:      "What is his age?"
```

If the LLM forgot previous messages, "his age" would be meaningless — *whose* age? So each message and its role are stored, letting the LLM understand **context**. Same reason "the code

you gave is wrong” works — the LLM knows *which* code was the assistant’s.

A conversation IS just a list of messages, each tagged with a role.

Response — what comes back?

The response is not just the answer — it’s a big object. Two important fields:

- `choices` → your **main answer**. Called “choices” because an LLM can be configured to return multiple answers (default is 1). Treat it as an *array of answers* → get the first with `choices[0]`.
- `usage` → how many **tokens** were used. A token $\approx$ ~4 characters / roughly one word. Longer input = more tokens. Groq is free (with a generous limit); OpenAI charges ~\$2 per 1M tokens. That’s why tokens matter.

Day 1 + 2 — Full Code (`hello_llm.py`)

```
import os
from pathlib import Path
from dotenv import load_dotenv
from groq import Groq

# 1. Load secrets from the .env file
load_dotenv()

# 2. Read the API key (env variable name is GROQ_API_KEY)
my_api_key = os.getenv("GROQ_API_KEY")

# 3. Safety check – stop the program if the key is missing
if not my_api_key:
    raise ValueError("API KEY not found")

# 4. Register as a client
client = Groq(api_key=my_api_key)

# 5. The EXACT model name
model = "llama-3.3-70b-versatile"

# 6. Build a message – role + content
prompt = "Do you know Padho With Pratyush?"
message = {
    "role": "user",
    "content": prompt          # the key must be "content", NOT "prompt"
}

# 7. Groq needs a LIST of messages (even if there's just one)
```

```
messages = [message]

# 8. Make the LLM call
response = client.chat.completions.create(
    model=model,
    messages=messages
)

# 9. Extract just the main answer
print(response.choices[0].message.content)
```

Install + run (Mac):

```
uv add groq python-dotenv      # add packages
uv run hello_llm.py            # 🍏 recommended – auto-uses the venv
# or, after activating the venv:
python3 hello_llm.py
```

🍏 **Mac note:** The video runs `python hello_llm.py` (Windows). On Mac use `python3 hello_llm.py`, or better `uv run hello_llm.py` so you never have to activate the venv manually.

Common error (from the video): the dictionary key must be `content`, not `prompt`. That's the API's rule — using `prompt` throws an error. (Naming the variable `prompt` is fine; the dict key is what matters.)

Debugging tip (from the video): ~90% of errors are human error. Read the error, find the line number, comment out suspect lines to isolate it. The computer rarely breaks — you usually mistyped something.